

*Pandas*模块的使用





目录

01 序列与数据框的介绍

02 外部数据的读取

03 数据的清洗

04 数据的合并与连接

05 数据的汇总

pandas

pandas is a fast, powerful, flexible and easy to use open source data analysis and manipulation tool, built on top of the [Python](#) programming language.

[Install pandas now!](#)

Getting started

- [Install pandas](#)
- [Getting started](#)

Documentation

- [User guide](#)
- [API reference](#)
- [Contributing to pandas](#)
- [Release notes](#)

Community

- [About pandas](#)
- [Ask a question](#)
- [Ecosystem](#)

With the support of:



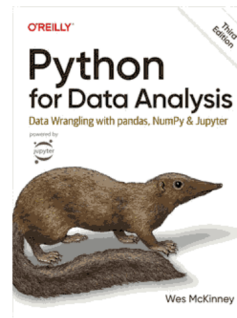
Latest version: 2.0.1

- [What's new in 2.0.1](#)
- Release date:
Apr 24, 2023
- [Documentation \(web\)](#)
- [Download source code](#)

Follow us



Get the book



Previous versions

- 2.0.0 (Apr 03, 2023)
[changelog](#) | [docs](#) | [code](#)

序列与数据框的介绍

序列(Series)与数据框(DataFrame)的定义

序列：是一种存储数据的容器，类似于一维数组，要求序列中的元素具有相同的数据类型。可以将其理解为数据集中的一个字段（列）。

数据框：同样是存储数据的容器，类似于二维数组，所不同的是，数据框中的每一列可以是不同的数据类型。数据框实际上是由多个序列组合而成的。

序列与数据框的介绍

序列构造的方法

- ✦ 通过同质的列表或元组构造
- ✦ 通过字典构造
- ✦ 通过Numpy中的一维数组构造
- ✦ 通过数据框DataFrame中的某一行构造

序列与数据框的介绍

序列构造的案例

```
# 导入模块
import pandas as pd
import numpy as np

# 构造序列
gdp1 = pd.Series([2.8,3.01,8.99,8.59,5.18])
gdp2 = pd.Series({'北京':2.8,'上海':3.01,'广东':8.99,'江苏':8.59,'浙江':5.18})
gdp3 = pd.Series(np.array((2.8,3.01,8.99,8.59,5.18)))

print(gdp1)
print(gdp2)
print(gdp3) # 打印结果同gdp1
```

out:

```
0    2.80
1    3.01
2    8.99
3    8.59
4    5.18
dtype: float64
```

```
上海    3.01
北京    2.80
广东    8.99
江苏    8.59
浙江    5.18
dtype: float64
```

序列与数据框的介绍

序列构造的说明

- 1、从序列的打印结果看，会包含两列内容，第一列属于序列的行索引（可以理解为行号），自动从0开始，第二列属于序列的实际值；
- 2、通过字典构造的序列仍然包含两列，所不同的是第一列不再是行号，而是行名称（label），其对应到字典中的键，第二列是序列的实际值，对应到字典中的值；

序列与数据框的介绍

序列元素的返回方法

- ✦ 位置索引法：该用法与字符串中的索引或切片完全一致，除此还可以使用任意不连续的位置索引
- ✦ 名称索引法：如果序列是通过字典构造，可以使用名称索引法取出对应的元素
- ✦ 布尔索引法：该用法是结合条件表达式将序列中的元素取出

序列与数据框的介绍

序列元素返回的案例

```
# 取出序列gdp1中的第一、第四和第五个元素  
print(gdp1[[0,3,4]])
```

```
# 取出序列gdp2中上海、江苏和浙江的GDP值  
print(gdp2[['上海','江苏','浙江']])
```

```
# 取出序列gdp2中，GDP超过5.0的元素  
print(gdp2[gdp2>5.0])
```

```
# 数学函数--取对数  
print('通过numpy函数 : \n',np.log(gdp1))
```

```
# 平均gdp  
print('通过numpy函数 : \n',np.mean(gdp1))  
print('通过序列的方法 : \n',gdp1.mean())
```

out:

0	2.80	通过numpy函数 :
3	8.59	0 1.029619
4	5.18	1 1.101940
dtype: float64		2 2.196113
		3 2.150599
上海	3.01	4 1.644805
江苏	8.59	dtype: float64
浙江	5.18	通过numpy函数 :
dtype: float64		5.714
广东	8.99	
江苏	8.59	通过序列的方法 :
浙江	5.18	5.714
dtype: float64		

序列与数据框的介绍

数据框构造的方法

- ✦ 通过嵌套的列表或元组构造
- ✦ 通过字典构造
- ✦ 通过二维数组构造
- ✦ 通过外部数据的读取构造

序列与数据框的介绍

数据框构造的案例

基于嵌套的列表

```
df1 = pd.DataFrame([['张三',23,'男'],['李四',27,'女'],['王二',26,'女']])
```

基于字典

```
df2 = pd.DataFrame({'姓名':['张三','李四','王二'],'年龄':[23,27,26],'性别':['男','女','女']})
```

基于数组

```
df3 = pd.DataFrame(np.array([['张三',23,'男'],['李四',27,'女'],['王二',26,'女']]))
```

```
print('嵌套列表构造数据框：\n',df1)
```

```
print('字典构造数据框：\n',df2)
```

```
print('二维数组构造数据框：\n' ,df3) # 输出结果同df1
```

out:

嵌套列表构造数据框：

	0	1	2
0	张三	23	男
1	李四	27	女
2	王二	26	女

字典构造数据框：

	姓名	年龄	性别
0	张三	23	男
1	李四	27	女
2	王二	26	女

二维数组构造数据框：

	0	1	2
0	张三	23	男
1	李四	27	女
2	王二	26	女

外部数据的读取

文本文件的读取

```
pd.read_table(filepath_or_buffer, sep='\t', header='infer', names=None, usecols=None,
               skiprows=None, skipfooter=None, comment=None, encoding=None,
               parse_dates=False, thousands=None)
```

filepath_or_buffer : 指定txt文件或csv文件所在的具体路径

sep : 指定原数据集中各字段之间的分隔符，默认为Tab制表符

header : 是否需要将原数据集中的第一行作为表头，默认将第一行用作字段名称

names : 如果原数据集中没有字段，可以通过该参数在数据读取时给数据框添加具体的表头

usecols : 指定需要读取原数据集中的哪些变量名

skiprows : 数据读取时，指定需要跳过原数据集开头的行数

外部数据的读取

文本文件的读取

```
pd.read_table(filepath_or_buffer, sep='\t', header='infer', names=None, usecols=None,
               skiprows=None, skipfooter=None, comment=None, encoding=None,
               parse_dates=False, thousands=None)
```

skipfooter : 数据读取时，指定需要跳过原数据集末尾的行数

comment : 指定注释符，在读取数据时，如果碰到行首指定的注释符，则跳过该行

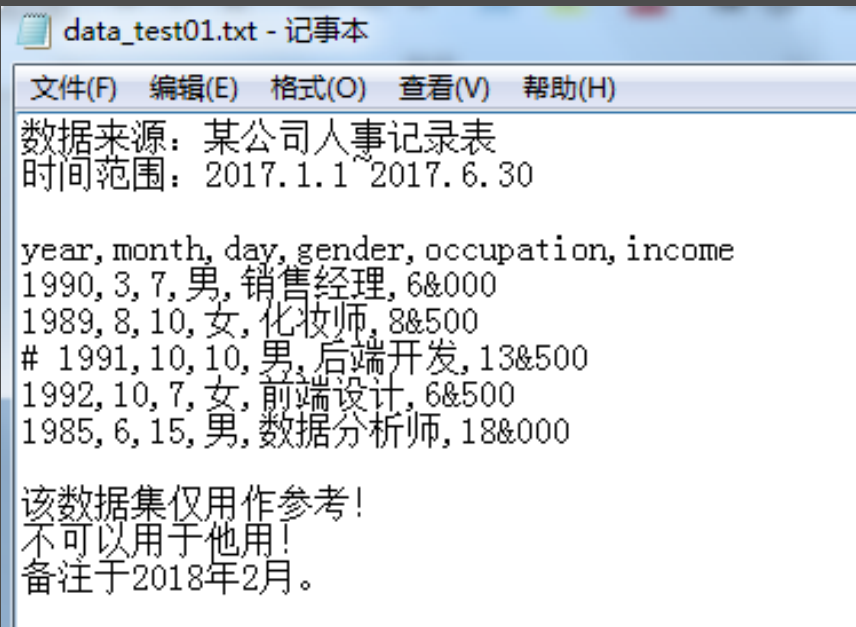
encoding : 如果文件中含有中文，有时需要指定字符编码

parse_dates : 如果参数值为True，则尝试解析数据框的行索引；如果参数为列表，则尝试解析对应的日期列；如果参数为嵌套列表，则将某些列合并为日期列；如果参数为字典，则解析对应的列（字典中的值），并生成新的字段名（字典中的键）。

thousands : 指定原始数据集中的千分位符

外部数据的读取

文本文件的读取案例



```
data_test01.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

数据来源: 某公司人事记录表
时间范围: 2017.1.1~2017.6.30

year,month,day,gender,occupation,income
1990,3,7,男,销售经理,6&000
1989,8,10,女,化妆师,8&500
# 1991,10,10,男,后端开发,13&500
1992,10,7,女,前端设计,6&500
1985,6,15,男,数据分析师,18&000

该数据集仅用作参考!
不可以用于他用!
备注于2018年2月。
```

- 数据集并不是从第一行开始，前面几行实际上是数据集的来源说明，读取数据时需要注意什么问题。
- 数据集的末尾3行仍然不是需要读入的数据，如何避免后3行数据的读入。
- 中间部分的数据，第四行前加了#号，表示不需要读取该行，该如何处理。
- 数据集集中的收入一列，千分位符是&，如何将该字段读入为正常的数值型数据。
- 如果需要将year、month和day三个字段解析为新的birthday字段，该如何做到。
- 数据集集中含有中文，一般在读取含中文的文本文件时都会出现编码错误，该如何解决

外部数据的读取

文本文件的读取案例

读取文本文件中的数据

```
user_income = pd.read_table(r'C:\Users\Administrator\Desktop\data_test01.txt',  
                             sep = ',',  
                             parse_dates={'birthday':[0,1,2]},  
                             skiprows=2, skipfooter=3, comment='#',  
                             encoding='utf8', thousands='&')
```

	birthday	gender	occupation	income
0	1990-03-07	男	销售经理	6000
1	1989-08-10	女	化妆师	8500
2	1992-10-07	女	前端设计	6500
3	1985-06-15	男	数据分析师	18000

外部数据的读取

电子表格的读取

```
pd.read_excel(io, sheetname=0, header=0, skiprows=None, skip_footer=0,  
              index_col=None, names=None, parse_cols=None, parse_dates=False,  
              na_values=None, thousands=None, convert_float=True)
```

io : 指定电子表格的具体路径

sheetname : 指定需要读取电子表格中的第几个Sheet，既可以传递整数也可以传递具体的Sheet名称

header : 是否需要将数据集的第一行用作表头，默认为需要的

skiprows : 读取数据时，指定跳过的开始行数

skip_footer : 读取数据时，指定跳过的末尾行数

index_col : 指定哪些列用作数据框的行索引（标签）

names : 如果原数据集中没有字段，可以通过该参数在数据读取时给数据框添加具体的表头

外部数据的读取

电子表格的读取

```
pd.read_excel(io, sheetname=0, header=0, skiprows=None, skip_footer=0,  
              index_col=None, names=None, parse_cols=None, parse_dates=False,  
              na_values=None, thousands=None, convert_float=True)
```

parse_cols : 指定需要解析的字段

parse_dates : 如果参数值为True，则尝试解析数据框的行索引；如果参数为列表，则尝试解析对应的日期列；如果参数为嵌套列表，则将某些列合并为日期列；如果参数为字典，则解析对应的列（字典中的值），并生成新的字段名（字典中的键）

na_values : 指定原始数据中哪些特殊值代表了缺失值

thousands : 指定原始数据集中的千分位符

convert_float : 默认将所有的数值型字段转换为浮点型字段

converters : 通过字典的形式，指定某些列需要转换的形式

外部数据的读取

电子表格读取案例

	A	B	C	D
1	00101	儿童裤	黑色	109
2	01123	儿童上衣	红色	229
3	01010	儿童鞋	蓝色	199
4	00100	儿童内衣	灰色	159

- 该表没有表头，如何读数据的同时就设置好具体的表头。
- 数据集的第一列实际上是字符型的字段，如何避免数据读入时自动变成数值型字段。

外部数据的读取

电子表格读取案例

读取电子表格数据

```
child_cloth = pd.read_excel(io = r'C:\Users\Administrator\Desktop\data_test02.xlsx',  
                             header = None, converters = {0:str}  
                             names = ['Prod_Id','Prod_Name','Prod_Color','Prod_Price'])
```

child_cloth

	Prod_Id	Prod_Name	Prod_Color	Prod_Price
0	00101	儿童裤	黑色	109
1	01123	儿童上衣	红色	229
2	01010	儿童鞋	蓝色	199
3	00100	儿童内衣	灰色	159

数据的清洗

数据类型的转换

- 对于数值型、字符型、布尔型等类型的转换可以使用`astype`方法；
- 对于日期时间型的转换需使用`to_datetime`函数；

数据的清洗

数据类型的转换

	Brand	Name	Boarding_time	Km(W)	Discharge	Sec_price	New_price
0	众泰	众泰T600 2016款 1.5T 手动 豪华型	2016年5月	3.96	国4	6.8	9.42万
1	众泰	众泰Z700 2016款 1.8T 手动 典雅型	2017年8月	0.08	国4,国5	8.8	11.92万
2	众泰	大迈X5 2015款 1.5T 手动 豪华型	2016年9月	0.80	国4	5.8	8.56万
3	众泰	众泰T600 2017款 1.5T 手动 精英贺岁版	2017年3月	0.30	国5	6.2	8.66万
4	众泰	众泰T600 2016款 1.5T 手动 旗舰型	2016年2月	1.70	国4	7.0	11.59万

- 二手车的上牌时间Boarding_time应该为日期型；
- 新车价格New_price应该为浮点型；

数据的清洗

数据类型的转换

```
# 数据读取
sec_cars = pd.read_table(r'C:\Users\Administrator\Desktop\sec_cars.csv', sep = ',')

# 修改二手车上牌时间的数据类型
sec_cars.Boarding_time = pd.to_datetime(sec_cars.Boarding_time, format = '%Y年%m月')
# 修改二手车新车价格的数据类型
sec_cars.New_price = sec_cars.New_price.str[:-1].astype('float')

# 重新查看各变量数据类型
sec_cars.dtypes
```

out:

```
out:
Brand           object
Name            object
Boarding_time   datetime64[ns]
Km(W)           float64
Discharge       object
Sec_price       float64
New_price       float64
dtype: object
```

数据的清洗

重复观察的识别与处理

	A	B	C	D	E	F	G
1	appcategory	appname	comments	install	love	size	update
2	网上购物-商城-团购-优惠-快递	每日优鲜	1297	204.7万	89.00%	15.16MB	2017年10月11日
3	网上购物-商城	苏宁易购	577	7996.8万	73.00%	58.9MB	2017年09月21日
4	网上购物-商城-优惠	唯品会	2543	7090.1万	86.00%	41.43MB	2017年10月13日
5	网上购物-商城-优惠	唯品会	2543	7090.1万	86.00%	41.43MB	2017年10月13日
6	网上购物-商城	拼多多	1921	3841.9万	95.00%	13.35MB	2017年10月11日
7	网上购物-商城-优惠	寺库奢侈品	1964	175.4万	100.00%	17.21MB	2017年09月30日
8	网上购物-商城	淘宝	14244	4.6亿	68.00%	73.78MB	2017年10月13日
9	网上购物-商城-团购-优惠	当当	134	1615.3万	61.00%	37.01MB	2017年10月17日
10	网上购物-商城-团购-优惠	当当	134	1615.3万	61.00%	37.01MB	2017年10月17日
11	网上购物-商城-团购-优惠	当当	134	1615.3万	61.00%	37.01MB	2017年10月17日

duplicated方法用于重复观测的识别

数据的清洗

重复观察的识别与处理

```
# 数据读入
df = pd.read_excel(r'C:\Users\Administrator\Desktop\data_test04.xlsx')
# 重复观测的检测
print('数据集中是否存在重复观测：\n', any(df.duplicated()))
```

out:
数据集中是否存在重复观测：
True

```
# 删除重复项
df.drop_duplicates(inplace = True)
df
```

	appcategory	appname	comments	install	love	size	update
0	网上购物-商城-团购-优惠-快递	每日优鲜	1297	204.7万	89.00%	15.16MB	2017年10月11日
1	网上购物-商城	苏宁易购	577	7996.8万	73.00%	58.9MB	2017年09月21日
2	网上购物-商城-优惠	唯品会	2543	7090.1万	86.00%	41.43MB	2017年10月13日
4	网上购物-商城	拼多多	1921	3841.9万	95.00%	13.35MB	2017年10月11日
5	网上购物-商城-优惠	寺库奢侈品	1964	175.4万	100.00%	17.21MB	2017年09月30日
6	网上购物-商城	淘宝	14244	4.6亿	68.00%	73.78MB	2017年10月13日
7	网上购物-商城-团购-优惠	当当	134	1615.3万	61.00%	37.01MB	2017年10月17日

数据的清洗

缺失值的识别与处理

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.0	6500.0
1	81201135	2016-11-08	M	27.0	10300.0
2	80043782	2016-10-13	F	NaN	13500.0
3	84639281	2017-04-17	M	26.0	6000.0
4	73499801	2016-03-21	NaN	NaN	4500.0
5	72399510	2016-01-18	M	19.0	NaN
6	63881943	2015-10-07	M	21.0	10000.0
7	35442690	2015-04-10	F	NaN	5800.0
8	77638351	2016-07-12	M	25.0	18000.0
9	85200189	2017-05-18	M	22.0	NaN

isnull方法用于缺失观测的识别

数据的清洗

缺失值的识别与处理

数据读入

```
df = pd.read_excel(r'C:\Users\Administrator\Desktop\data_test05.xlsx')
```

缺失观测的检测

```
print('数据集中是否存在缺失值：\n',any(df.isnull()))
```

out:

数据集中是否存在缺失值：
True

删除法之记录删除

```
df.dropna()
```

删除法之变量删除

```
df.drop('age', axis = 1)
```

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.0	6500.0
1	81201135	2016-11-08	M	27.0	10300.0
3	84639281	2017-04-17	M	26.0	6000.0
6	63881943	2015-10-07	M	21.0	10000.0
8	77638351	2016-07-12	M	25.0	18000.0

	uid	regit_date	gender	income
0	81200457	2016-10-30	M	6500.0
1	81201135	2016-11-08	M	10300.0
2	80043782	2016-10-13	F	13500.0
3	84639281	2017-04-17	M	6000.0
4	73499801	2016-03-21	NaN	4500.0
5	72399510	2016-01-18	M	NaN
6	63881943	2015-10-07	M	10000.0
7	35442690	2015-04-10	F	5800.0
8	77638351	2016-07-12	M	18000.0
9	85200189	2017-05-18	M	NaN

数据的清洗

缺失值的识别与处理

替换法之前向替换

`df.fillna(method = 'ffill' )`

替换法之后向替换

`df.fillna(method = 'bfill')`

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.0	6500.0
1	81201135	2016-11-08	M	27.0	10300.0
2	80043782	2016-10-13	F	27.0	13500.0
3	84639281	2017-04-17	M	26.0	6000.0
4	73499801	2016-03-21	M	26.0	4500.0
5	72399510	2016-01-18	M	19.0	4500.0
6	63881943	2015-10-07	M	21.0	10000.0
7	35442690	2015-04-10	F	21.0	5800.0
8	77638351	2016-07-12	M	25.0	18000.0
9	85200189	2017-05-18	M	22.0	18000.0

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.0	6500.0
1	81201135	2016-11-08	M	27.0	10300.0
2	80043782	2016-10-13	F	26.0	13500.0
3	84639281	2017-04-17	M	26.0	6000.0
4	73499801	2016-03-21	M	19.0	4500.0
5	72399510	2016-01-18	M	19.0	10000.0
6	63881943	2015-10-07	M	21.0	10000.0
7	35442690	2015-04-10	F	25.0	5800.0
8	77638351	2016-07-12	M	25.0	18000.0
9	85200189	2017-05-18	M	22.0	NaN

数据的清洗

缺失值的识别与处理

替换法之常数替换

```
df.fillna(value = 0)
```

替换法之统计值替换

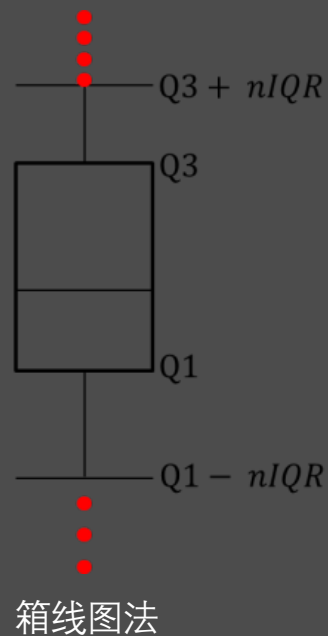
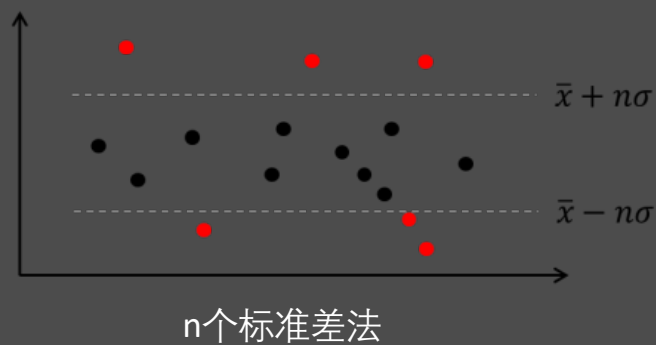
```
df.fillna(value = {'gender':df.gender.mode()[0],  
                  'age':df.age.mean(),  
                  'income':df.income.median()})
```

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.0	6500.0
1	81201135	2016-11-08	M	27.0	10300.0
2	80043782	2016-10-13	F	0.0	13500.0
3	84639281	2017-04-17	M	26.0	6000.0
4	73499801	2016-03-21	0	0.0	4500.0
5	72399510	2016-01-18	M	19.0	0.0
6	63881943	2015-10-07	M	21.0	10000.0
7	35442690	2015-04-10	F	0.0	5800.0
8	77638351	2016-07-12	M	25.0	18000.0
9	85200189	2017-05-18	M	22.0	0.0

	uid	regit_date	gender	age	income
0	81200457	2016-10-30	M	23.000000	6500.0
1	81201135	2016-11-08	M	27.000000	10300.0
2	80043782	2016-10-13	F	23.285714	13500.0
3	84639281	2017-04-17	M	26.000000	6000.0
4	73499801	2016-03-21	M	23.285714	4500.0
5	72399510	2016-01-18	M	19.000000	8250.0
6	63881943	2015-10-07	M	21.000000	10000.0
7	35442690	2015-04-10	F	23.285714	5800.0
8	77638351	2016-07-12	M	25.000000	18000.0
9	85200189	2017-05-18	M	22.000000	8250.0

数据的清洗

异常值的识别与处理



数据的清洗

异常值的识别与处理

```
# 数据读入
sunspots = pd.read_table(r'C:\Users\Administrator\Desktop\sunspots.csv', sep = ',')
# 异常值检测之标准差法
xbar = sunspots.counts.mean()
xstd = sunspots.counts.std()
print('标准差法异常值上限检测 : \n',any(sunspots.counts > xbar + 2 * xstd))
print('标准差法异常值下限检测 : \n',any(sunspots.counts < xbar - 2 * xstd))

# 异常值检测之箱线图法
Q1 = sunspots.counts.quantile(q = 0.25)
Q3 = sunspots.counts.quantile(q = 0.75)
IQR = Q3 - Q1
print('箱线图法异常值上限检测 : \n',any(sunspots.counts > Q3 + 1.5 * IQR))
print('箱线图法异常值下限检测 : \n',any(sunspots.counts < Q1 - 1.5 * IQR))
```

out:
True
False
True
False

数据的清洗

异常值的识别与处理

```
# 替换法处理异常值
print('异常值替换前的数据统计特征：\n',sunspots.counts.describe())
# 箱线图上的异常值判别上限
UL = Q3 + 1.5 * IQR
print('判别异常值的上限临界值：\n',UL)
# 从数据中找出低于判别上限的最大值
replace_value = sunspots.counts[sunspots.counts < UL].max()
print('用以替换异常值的数据：\n',replace_value)
# 替换超过判别上限异常值
sunspots.counts[sunspots.counts > UL] = replace_value
print('异常值替换后的数据统计特征：\n',sunspots.counts.describe())
```

out:

判别异常值的上限临界值：148.85
用以替换异常值的数据：141.7

数据的清洗

数据子集的筛选

- ✦ `iloc`只能通过行号和列号进行数据的筛选
- ✦ `loc`要比`iloc`灵活一些，可以指定具体的行标签（或条件表达式）和列标签（字段名）
- ✦ `ix`是`iloc`和`loc`的混合，其吸收了`iloc`和`loc`的优点，使数据框子集的获取更加灵活

数据的清洗

数据子集的筛选

构造数据集

```
df1 = pd.DataFrame({'name':['张三','李四','王二','丁一','李五'],  
                    'gender':['男','女','女','女','男'],  
                    'age':[23,26,22,25,27]})
```

df1

取出数据集的中间三行(即所有女性), 并且返回姓名和年龄两列

```
df1.iloc[1:3,[0,2]]
```

```
df1.loc[1:3, ['name','age']]
```

```
df1.ix[1:3,[0,2]]
```

	name	gender	age
0	张三	男	23
1	李四	女	26
2	王二	女	22
3	丁一	女	25
4	李五	男	27

	name	age
1	李四	26
2	王二	22
3	丁一	25

	name	age
1	李四	26
2	王二	22
3	丁一	25

	name	age
1	李四	26
2	王二	22
3	丁一	25

数据的清洗

数据子集的筛选

使用筛选条件，取出所有男性的姓名和年龄

```
# df1.iloc[df1.gender == '男',]  
df1.loc[df1.gender == '男',['name','age']]  
df1.ix[df1.gender == '男',['name','age']]
```

	name	gender	age
0	张三	男	23
1	李四	女	26
2	王二	女	22
3	丁一	女	25
4	李五	男	27

	name	age
0	张三	23
4	李五	27

	name	age
0	张三	23
4	李五	27

数据的汇总

Excel中的透视表

	A	B	C	D	E	F
1						
2						
3	行标签	平均值项:price				
4	D	3169.954096				
5	E	3076.752475				
6	F	3724.886397				
7	G	3999.135671				
8	H	4486.669196				
9	I	5091.874954				
10	J	5323.81802				
11	总计	3932.799722				
12						
13						

数据透视表字段列表

选择要添加到报表的字段:

depth

在以下区域间拖动字段:

报表筛选

行标签

列标签

Σ 数值

color 平均值项:price

推迟布局更新

更新

	A	B	C	D	E	F	G	H	I
1									
2									
3	计数项:carat	列标签							
4	行标签	Fair	Good	Ideal	Premium	Very Good	总计		
5	I1	210	96	146	205	84	741		
6	IF	9	71	1212	230	268	1790		
7	SI1	408	1560	4282	3575	3240	13065		
8	SI2	466	1081	2598	2949	2100	9194		
9	VS1	170	648	3589	1989	1775	8171		
10	VS2	261	978	5071	3357	2591	12258		
11	VVS1	17	186	2047	616	789	3655		
12	VVS2	69	286	2606	870	1235	5066		
13	总计	1610	4906	21551	13791	12082	53940		

数据透视表字段列表

选择要添加到报表的字段:

carat

在以下区域间拖动字段:

报表筛选

行标签

列标签

Σ 数值

clarity 计数项:carat

推迟布局更新

更新

数据的汇总

Python中的透视表

```
pd.pivot_table(data, values=None, index=None, columns=None, aggfunc='mean',  
               fill_value=None, margins=False, dropna=True, margins_name='All')
```

data : 指定需要构造透视表的数据集

values : 指定需要拉入“数值”框的字段列表

index : 指定需要拉入“行标签”框的字段列表

columns : 指定需要拉入“列标签”框的字段列表

aggfunc : 指定数值的统计函数，默认为统计均值，也可以指定numpy模块中的其他统计函数

fill_value : 指定一个标量，用于填充缺失值

margins : bool类型参数，是否需要显示行或列的总计值，默认为False

dropna : bool类型参数，是否需要删除整列为缺失的字段，默认为True

margins_name : 指定行或列的总计名称，默认为All

数据的汇总

Python中的透视表

数据读取

```
diamonds = pd.read_table(r'diamonds.csv', sep = ',')
```

单个分组变量的均值统计

```
pd.pivot_table(data = diamonds, index = 'color', values = 'price',  
               margins = True, margins_name = '总计')
```

diamonds										
carat	cut	color	clarity	depth	table	price	x	y	z	
0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43	
0.21	Premium	E	SI1	59.8	61	326	3.89	3.84	2.31	
0.23	Good	E	VS1	56.9	65	327	4.05	4.07	2.31	
0.29	Premium	I	VS2	62.4	58	334	4.2	4.23	2.63	
0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75	
0.24	Very Good	J	VVS2	62.8	57	336	3.94	3.96	2.48	
0.24	Very Good	I	VVS1	62.3	57	336	3.95	3.98	2.47	
0.26	Very Good	H	SI1	61.9	55	337	4.07	4.11	2.53	
0.22	Fair	E	VS2	65.1	61	337	3.87	3.78	2.49	
0.23	Very Good	H	VS1	59.4	61	338	4	4.05	2.39	
0.3	Good	J	SI1	64	55	339	4.25	4.28	2.73	
0.23	Ideal	J	VS1	62.8	56	340	3.93	3.9	2.46	
0.22	Premium	F	SI1	60.4	61	342	3.88	3.84	2.33	

```
color
D      3169.954096
E      3076.752475
F      3724.886397
G      3999.135671
H      4486.669196
I      5091.874954
J      5323.818020
总计    3932.799722
Name: price, dtype: float64
```

数据的汇总

Python中的透视表

```
# 两个分组变量的列联表  
# 导入numpy模块  
import numpy as np
```

```
pd.pivot_table(data = diamonds, index = 'clarity', columns = 'cut', values = 'carat',  
               aggfunc = np.size, margins = True, margins_name = '总计')
```

diamonds										
carat	cut	color	clarity	depth	table	price	x	y	z	
0.23	Ideal	E	SI2	61.5	55	326	3.95	3.98	2.43	
0.21	Premium	E	SI1	59.8	61	326	3.89	3.84	2.31	
0.23	Good	E	VS1	56.9	65	327	4.05	4.07	2.31	
0.29	Premium	I	VS2	62.4	58	334	4.2	4.23	2.63	
0.31	Good	J	SI2	63.3	58	335	4.34	4.35	2.75	
0.24	Very Good	J	VVS2	62.8	57	336	3.94	3.96	2.48	
0.24	Very Good	I	VVS1	62.3	57	336	3.95	3.98	2.47	
0.26	Very Good	H	SI1	61.9	55	337	4.07	4.11	2.53	
0.22	Fair	E	VS2	65.1	61	337	3.87	3.78	2.49	
0.23	Very Good	H	VS1	59.4	61	338	4	4.05	2.39	
0.3	Good	J	SI1	64	55	339	4.25	4.28	2.73	
0.23	Ideal	J	VS1	62.8	56	340	3.93	3.9	2.46	
0.22	Premium	F	SI1	60.4	61	342	3.88	3.84	2.33	

cut	Fair	Good	Ideal	Premium	Very Good	总计
clarity						
I1	210.0	96.0	146.0	205.0	84.0	741.0
IF	9.0	71.0	1212.0	230.0	268.0	1790.0
SI1	408.0	1560.0	4282.0	3575.0	3240.0	13065.0
SI2	466.0	1081.0	2598.0	2949.0	2100.0	9194.0
VS1	170.0	648.0	3589.0	1989.0	1775.0	8171.0
VS2	261.0	978.0	5071.0	3357.0	2591.0	12258.0
VVS1	17.0	186.0	2047.0	616.0	789.0	3655.0
VVS2	69.0	286.0	2606.0	870.0	1235.0	5066.0
总计	1610.0	4906.0	21551.0	13791.0	12082.0	53940.0

数据的汇总

Python中的透视表

James_Harden									
对手	胜负	主客场	命中	投篮数	投篮命中率	3分命中率	篮板	助攻	得分
勇士	胜	客	10	23	0.435	0.444	6	11	27
国王	胜	客	8	21	0.381	0.286	3	9	27
小牛	胜	主	10	19	0.526	0.462	3	7	29
灰熊	负	主	8	20	0.4	0.25	5	8	22
76人	胜	客	10	20	0.5	0.25	3	13	27
黄蜂	胜	客	8	18	0.444	0.4	10	11	27
灰熊	负	客	6	19	0.316	0.222	4	8	20
76人	负	主	8	21	0.381	0.429	4	7	29
尼克斯	胜	客	9	23	0.391	0.353	5	9	31
老鹰	胜	客	8	15	0.533	0.545	3	11	29
爵士	胜	主	19	25	0.76	0.875	2	13	56

数据的汇总

Python中的透视表

```
df = pd.read_csv('James_Harden.csv',encoding='utf8')  
df.tail()
```

	对手	胜负	主客场	命中	投篮数	投篮命中率	3分命中率	篮板	助攻	得分
20	步行者	胜	主	8	22	0.364	0.333	8	10	29
21	湖人	胜	客	13	22	0.591	0.444	4	9	36
22	爵士	胜	客	8	19	0.421	0.333	5	3	29
23	开拓者	胜	客	16	29	0.552	0.571	8	3	48
24	鹈鹕	胜	主	8	16	0.500	0.400	1	17	26

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index=[u'对手'])
```

	3分命中率	助攻	命中	得分	投篮命中率	投篮数	篮板
对手							
76人	0.33950	10.00	9.0	28.00	0.4405	20.5	3.5
勇士	0.44400	11.00	10.0	27.00	0.4350	23.0	6.0
国王	0.28600	9.00	8.0	27.00	0.3810	21.0	3.0
太阳	0.54500	7.00	12.0	48.00	0.5450	22.0	2.0
小牛	0.46200	7.00	10.0	29.00	0.5260	19.0	3.0
尼克斯	0.36900	9.50	10.5	34.00	0.4175	25.0	3.5
开拓者	0.57100	3.00	16.0	48.00	0.5520	29.0	8.0
掘金	0.14300	9.00	6.0	21.00	0.3750	16.0	8.0
步行者	0.29150	12.50	8.5	27.50	0.3965	21.5	6.5
湖人	0.44400	9.00	13.0	36.00	0.5910	22.0	4.0
灰熊	0.35025	7.75	8.5	27.25	0.4015	21.0	4.5
爵士	0.60400	8.00	13.5	42.50	0.5905	22.0	3.5
猛龙	0.27300	11.00	8.0	38.00	0.3200	25.0	6.0

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index=[u'对手',u'主客场'])
```

		3分命中率	助攻	命中	得分	投篮命中率	投篮数	篮板
对手	主客场							
76人	主	0.4290	7.0	8.0	29.0	0.381	21.0	4.0
	客	0.2500	13.0	10.0	27.0	0.500	20.0	3.0
勇士	客	0.4440	11.0	10.0	27.0	0.435	23.0	6.0
国王	客	0.2860	9.0	8.0	27.0	0.381	21.0	3.0
太阳	客	0.5450	7.0	12.0	48.0	0.545	22.0	2.0
小牛	主	0.4620	7.0	10.0	29.0	0.526	19.0	3.0
尼克斯	主	0.3850	10.0	12.0	37.0	0.444	27.0	2.0
	客	0.3530	9.0	9.0	31.0	0.391	23.0	5.0
开拓者	客	0.5710	3.0	16.0	48.0	0.552	29.0	8.0
掘金	主	0.1430	9.0	6.0	21.0	0.375	16.0	8.0
步行者	主	0.3330	10.0	8.0	29.0	0.364	22.0	8.0
	客	0.2500	15.0	9.0	26.0	0.429	21.0	5.0
湖人	客	0.4440	9.0	13.0	36.0	0.591	22.0	4.0

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index  
=[u'主客场',u'对手'])
```

		3分命中率	助攻	命中	得分	投篮命中率	投篮数	篮板
主客场	对手							
主	76人	0.4290	7.0	8.0	29.0	0.381	21.0	4.0
	小牛	0.4620	7.0	10.0	29.0	0.526	19.0	3.0
	尼克斯	0.3850	10.0	12.0	37.0	0.444	27.0	2.0
	掘金	0.1430	9.0	6.0	21.0	0.375	16.0	8.0
	步行者	0.3330	10.0	8.0	29.0	0.364	22.0	8.0
	灰熊	0.3395	8.0	9.5	30.0	0.420	22.5	4.5
	爵士	0.8750	13.0	19.0	56.0	0.760	25.0	2.0
	猛龙	0.2730	11.0	8.0	38.0	0.320	25.0	6.0
	篮网	0.6150	8.0	13.0	37.0	0.650	20.0	10.0
	骑士	0.4290	13.0	8.0	35.0	0.381	21.0	11.0
	鹈鹕	0.4000	17.0	8.0	26.0	0.500	16.0	1.0

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index=[u'主客场',u'胜负' ],  
               values=[u'得分',u'助攻',u'篮板'])
```

		助攻	得分	篮板
主客场	胜负			
主	胜	10.555556	34.222222	5.444444
	负	8.666667	29.666667	5.000000
客	胜	9.000000	32.000000	4.916667
	负	8.000000	20.000000	4.000000

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index=[u'主客场',u'胜负' ],  
               values=[u'得分',u'助攻',u'篮板' ],  
               aggfunc=[np.sum,np.mean])
```

		sum			mean		
		助攻	得分	篮板	助攻	得分	篮板
主客场	胜负						
主	胜	95	308	49	10.555556	34.222222	5.444444
	负	26	89	15	8.666667	29.666667	5.000000
客	胜	108	384	59	9.000000	32.000000	4.916667
	负	8	20	4	8.000000	20.000000	4.000000

数据的汇总

Python中的透视表

```
pd.pivot_table(df,index=[u'主客场' ],  
               columns=[u'对手'],values=[u'得分' ],  
               aggfunc=[np.sum],fill_value=0,margins=1)
```

sum																			
得分																			
对手	76人	勇士	国王	太阳	小牛	尼克斯	开拓者	掘金	步行者	湖人	灰熊	爵士	猛龙	篮网	老鹰	骑士	鹈鹕	黄蜂	All
主客场																			
主	29.0	0.0	0.0	0.0	29.0	37.0	0.0	21.0	29.0	0.0	60.0	56.0	38.0	37.0	0.0	35.0	26.0	0.0	397.0
客	27.0	27.0	27.0	48.0	0.0	31.0	48.0	0.0	26.0	36.0	49.0	29.0	0.0	0.0	29.0	0.0	0.0	27.0	404.0
All	56.0	27.0	27.0	48.0	29.0	68.0	48.0	21.0	55.0	36.0	109.0	85.0	38.0	37.0	29.0	35.0	26.0	27.0	801.0

数据的汇总

Python中的透视表

```
table=pd.pivot_table(df,index=[u'对手',u'胜负' ],  
                      columns=[u'主客场'],values=[u'得分',u'助攻',u'篮板' ],  
                      aggfunc=[np.mean],fill_value=0)
```

		mean					
		助攻		得分		篮板	
		主	客	主	客	主	客
对手	胜负						
76人	胜	0	13	0	27	0	3
	负	7	0	29	0	4	0
勇士	胜	0	11	0	27	0	6
国王	胜	0	9	0	27	0	3
太阳	胜	0	7	0	48	0	2
小牛	胜	7	0	29	0	3	0
尼克斯	胜	10	9	37	31	2	5
开拓者	胜	0	3	0	48	0	8

数据的合并与连接

合并与连接的示意图

姓名	年龄	性别
张三	21	男
李四	25	女
王二	22	男

姓名	年龄	性别
丁一	23	女
赵五	20	女



姓名	年龄	性别
张三	21	男
李四	25	女
王二	22	男
丁一	23	女
赵五	20	女

行合并

ID	年龄
1	21
2	25
3	22
4	23

ID	科目	成绩
1	科目1	86
2	科目1	77
2	科目2	89
4	科目1	81



ID	年龄	科目	成绩
1	21	科目1	86
2	25	科目1	77
2	25	科目2	89
3	22	NaN	NaN
4	23	科目1	81

列连接

数据的合并与连接

数据的合并

```
pd.concat(objs, axis=0, join='outer', join_axes=None,  
          ignore_index=False, keys=None)
```

objs : 指定需要合并的对象，可以是序列、数据框或面板数据构成的列表

axis : 指定数据合并的轴，默认为0，表示合并多个数据的行，如果为1，就表示合并多个数据的列

join : 指定合并的方式，默认为outer，表示合并所有数据，如果改为inner，表示合并公共部分的数据

join_axes : 合并数据后，指定保留的数据轴

ignore_index : bool类型的参数，表示是否忽略原数据集的索引，默认为False，如果设为True，就表示忽略原索引并生成新索引

keys : 为合并后的数据添加新索引，用于区分各个数据部分

数据的合并与连接

数据的合并

```
# 构造数据集df1和df2
df1 = pd.DataFrame({'name':['张三','李四','王二'], 'age':[21,25,22],
                    'gender':['男','女','男']})
df2 = pd.DataFrame({'name':['丁一','赵五'], 'age':[23,22], 'gender':['女','女']})
# 数据集的纵向合并
pd.concat([df1,df2], keys = ['df1','df2'])

# 如果df2数据集中的“姓名变量为Name”
df2 = pd.DataFrame({'Name':['丁一','赵五'], 'age':[23,22], 'gender':['女','女']})
# 数据集的纵向合并
pd.concat([df1,df2])
```

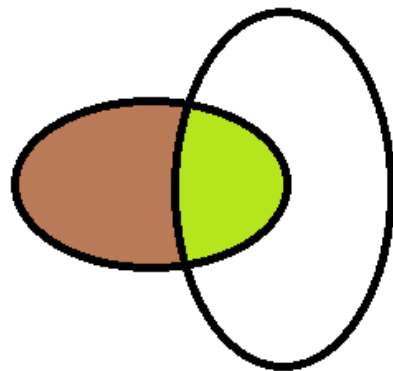
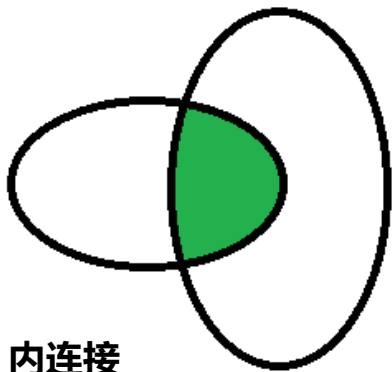
		age	gender	name
df1	0	21	男	张三
	1	25	女	李四
	2	22	男	王二
df2	0	23	女	丁一
	1	22	女	赵五

	Name	age	gender	name
0	NaN	21	男	张三
1	NaN	25	女	李四
2	NaN	22	男	王二
0	丁一	23	女	NaN
1	赵五	22	女	NaN

数据的合并与连接

数据的连接

- ✦ 内连接：用于返回两张表中的共同部分，可理解为集合的交集（缺点：两张表的数据都会有丢失）
- ✦ 左连接：保全左表的所有数据，将右表数据分配到左表中



数据的合并与连接

数据的连接

```
pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None,  
         left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'))
```

left : 指定需要连接的主表

right : 指定需要连接的辅表

how : 指定连接方式，默认为inner内连，还有其他选项，如左连left、右连right和外连outer

on : 指定连接两张表的共同字段

数据的合并与连接

数据的连接

```
pd.merge(left, right, how='inner', on=None, left_on=None, right_on=None,  
         left_index=False, right_index=False, sort=False, suffixes=('_x', '_y'))
```

left_on : 指定主表中需要连接的共同字段

right_on : 指定辅表中需要连接的共同字段

left_index : bool类型参数, 是否将主表中的行索引用作表连接的共同字段, 默认为False

right_index : bool类型参数, 是否将辅表中的行索引用作表连接的共同字段, 默认为False

sort : bool类型参数, 是否对连接后的数据按照共同字段排序, 默认为False

suffixes : 如果数据连接的结果中存在重叠的变量名, 则使用各自的前缀进行区分

数据的合并与连接

数据的连接

```
# 构造数据集
df3 = pd.DataFrame({'id':[1,2,3,4,5], 'name':['张三','李四','王二','丁一','赵五'],
                    'age':[27,24,25,23,25], 'gender':['男','男','男','女','女']})
df4 = pd.DataFrame({'Id':[1,2,2,4,4,4,5], 'score':[83,81,87,75,86,74,88]
                    'kemu':['科目1','科目1','科目2','科目1','科目2','科目3','科目1']})
df5 = pd.DataFrame({'id':[1,3,5], 'name':['张三','王二','赵五'],
                    'income':[13500,18000,15000]})
```

	age	gender	id	name
0	27	男	1	张三
1	24	男	2	李四
2	25	男	3	王二
3	23	女	4	丁一
4	25	女	5	赵五

	Id	kemu	score
0	1	科目1	83
1	2	科目1	81
2	2	科目2	87
3	4	科目1	75
4	4	科目2	86
5	4	科目3	74
6	5	科目1	88

	id	income	name
0	1	13500	张三
1	3	18000	王二
2	5	15000	赵五

数据的合并与连接

数据的连接

三表的数据连接

首先df3和df4连接

```
merge1 = pd.merge(left = df3, right = df4, how = 'left', left_on='id', right_on='Id')
```

merge1

再将连接结果与df5连接

```
merge2 = pd.merge(left = merge1, right = df5, how = 'left')
```

merge2

	age	gender	id	name	Id	kemu	score
0	27	男	1	张三	1.0	科目1	83.0
1	24	男	2	李四	2.0	科目1	81.0
2	24	男	2	李四	2.0	科目2	87.0
3	25	男	3	王二	NaN	NaN	NaN
4	23	女	4	丁一	4.0	科目1	75.0
5	23	女	4	丁一	4.0	科目2	86.0
6	23	女	4	丁一	4.0	科目3	74.0
7	25	女	5	赵五	5.0	科目1	88.0

	age	gender	id	name	Id	kemu	score	income
0	27	男	1	张三	1.0	科目1	83.0	13500.0
1	24	男	2	李四	2.0	科目1	81.0	NaN
2	24	男	2	李四	2.0	科目2	87.0	NaN
3	25	男	3	王二	NaN	NaN	NaN	18000.0
4	23	女	4	丁一	4.0	科目1	75.0	NaN
5	23	女	4	丁一	4.0	科目2	86.0	NaN
6	23	女	4	丁一	4.0	科目3	74.0	NaN
7	25	女	5	赵五	5.0	科目1	88.0	15000.0